

DeductIT User Documentation

Author: Rory O'Donoghue

Programme: Computer Science and Software Engineering

Project: Final Year Project

Supervisor: Rosemary Monahan

Academic Year: 2025/2026

1. Introduction

DeductIT is an interactive educational web-based tool designed to help students practise **natural deduction proofs**. The system provides a structured environment where users can construct logical proofs step by step while receiving automated feedback on correctness.

The application allows users to:

- Build logical formulas using symbol buttons
- Add proof lines
- Apply inference rules
- Create subproofs
- Automatically check completed proofs

The system supports two logic modes:

- **Propositional Logic**
- **Predicate Logic**

Users can switch between these modes depending on the type of logical exercises they wish to complete.

The goal of DeductIT is to provide an intuitive and guided environment that helps students learn and practise formal proof construction.

2. Running the System Locally

The DeductIT system consists of a **React frontend** and a **Node.js backend** proof checker. Both components must be running for the application to function correctly.

Steps to run the project locally

1. Open a terminal in the **frontend folder** and run:

```
npm install  
npm start
```

2. Open another terminal in the **backend folder** and run:

```
npm install  
npm start
```

The system should open itself in the browser but if not please open on `http://localhost:3000`

The frontend communicates with the backend proof checker at the following endpoint:

`http://localhost:5000/api/check-proof`

Important: Both the frontend and backend must be running simultaneously for the system to function correctly.

Requirements

To run the system locally, the user must have:

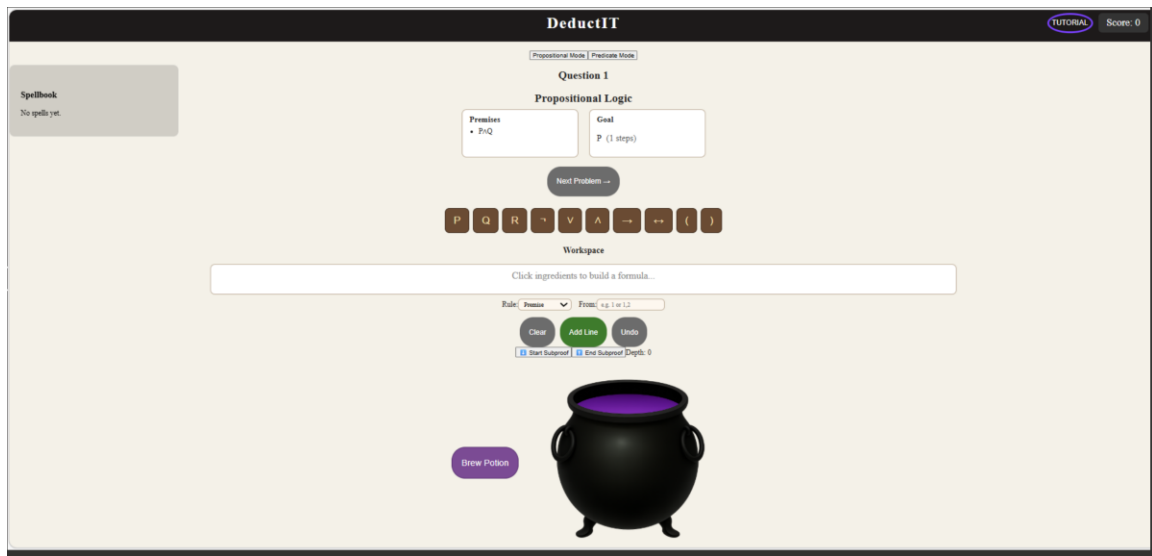
- **Node.js installed**
 - **npm installed**
 - **A modern web browser**, such as:
 - Google Chrome
 - Microsoft Edge
 - Mozilla Firefox
-

3. Interface Overview

The DeductIT interface is designed to guide users through the proof construction process. The main components of the interface include:

- **Header bar**
- **Tutorial button**
- **Score display**
- **Mode selector (Propositional / Predicate)**
- **Problem section** displaying premises and goal
- **Spellbook panel** containing proof lines
- **Ingredient buttons** for logical symbols
- **Workspace formula builder**
- **Rule selector**
- **From field** for referencing proof lines
- **Subproof controls**
- **Brew Potion** validation button

These components work together to allow the user to construct logical proofs step by step.



4. Buttons and Controls

The DeductIT interface includes a number of interactive controls used to build and verify proofs.

Tutorial

Opens a guided tutorial that highlights each section of the interface and explains its purpose.

Score Panel

Displays the user's current score. The score increases when a proof is completed successfully.

Propositional Mode

Loads exercises involving **propositional logic**.

Predicate Mode

Loads exercises involving **predicate logic**.

Next Problem

Loads the next exercise and clears the current proof.

Ingredient Buttons

Ingredient buttons allow the user to insert logical symbols used to build formulas.

Symbols include:

Propositional symbols

$P Q R \neg \vee \wedge \rightarrow \leftrightarrow ()$



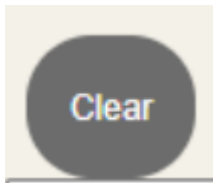
Additional Predicate logic symbols

$\forall \exists x a$



Clear

Clears the current formula in the workspace.



Add Line

Adds the current formula to the proof as a new line.



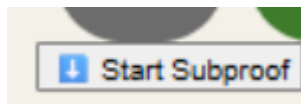
Undo

Removes the most recently added proof line.



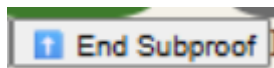
Start Subproof

Begins a new subproof and increases indentation.



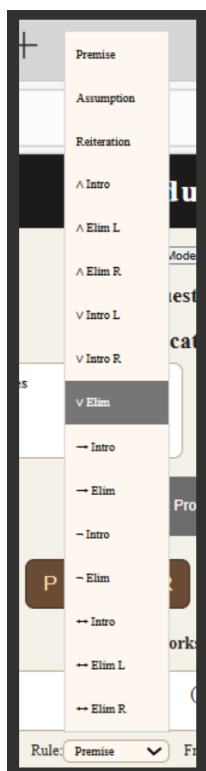
End Subproof

Closes the current subproof.



Rule Selector

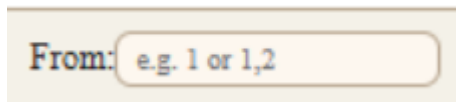
Allows the user to select the **inference rule** applied to the current proof line.



From Field

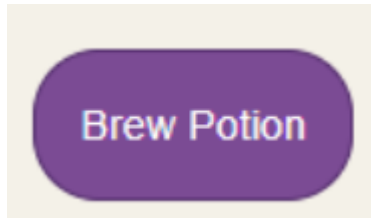
Specifies referenced proof lines. Example:

- From 1
- From 1,2



Brew Potion

Sends the proof to the backend proof checker for validation.



5. Building Logical Formulas

Formulas in DeductIT are constructed using **symbol buttons rather than free text input**.

This design ensures that all logical expressions remain syntactically valid.

The button-based formula builder prevents many common mistakes, including:

- Unbalanced parentheses
- Incorrect placement of logical connectives
- Invalid atomic symbols
- Malformed logical expressions
- Unsupported characters

Formulas appear in the **workspace display** before being added to the proof as a new line.

Once the formula is complete, the user can add it to the proof using the **Add Line** button.

6. Spellbook and Subproofs

The **Spellbook panel** displays the full proof constructed by the user.

Each proof line contains:

- **Line number**
- **Logical formula**
- **Rule used**
- **Referenced lines**

Assumptions appear with an **[assume] label** to indicate the start of a subproof.

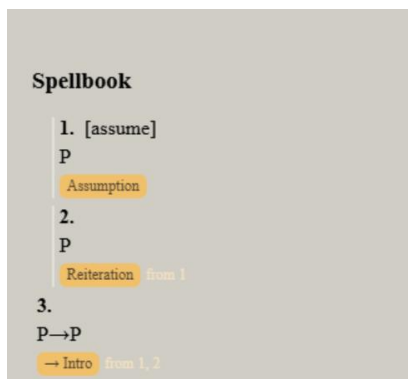
Subproofs

Subproofs allow the user to temporarily assume a statement in order to derive further conclusions.

Subproofs work as follows:

- **Start Subproof** creates a new assumption
- The proof lines inside the subproof are **indented**
- **End Subproof** closes the assumption block

Indentation visually represents the **scope of the assumption**.



7. Checking Proofs

The **Brew Potion** button validates the proof.

When clicked:

1. The proof is sent to the backend proof checker.
2. The backend analyses the proof structure.
3. All inference rules are validated.
4. The final proof line is compared with the goal formula.

Possible outcomes include:

Successful Proof

A success message is displayed when the proof is correct.

Potion Complete! Your proof is correct.

When a proof is successfully completed, the system increments the user's score counter.

Incorrect Proof

An error message appears when the proof does not satisfy the logical rules. The system will return information to the user about why the proof is incorrect.

✗ Potion Failed. Line 3: $\rightarrow$ Elim: conclusion must be B from A and $A \rightarrow B$.

8. Supported Natural Deduction Rules

The system supports a range of natural deduction inference rules.

Premise

A starting statement provided in the problem.

Assumption

Introduces a temporary assumption used to begin a subproof. This assumption can later be discharged when applying rules such as implication introduction.

Reiteration

Allows a previously derived statement to be repeated on a new line within the same scope or inside a subproof.

$\wedge$ Introduction

Combines two previously derived statements.

Combines two previously derived statements.

From P and Q , we may derive $P \wedge Q$.

$\wedge$ Elimination Left

Extracts the left component of a conjunction.

From $P \wedge Q$, we may derive P .

$\wedge$ Elimination Right

Extracts the right component of a conjunction.

From $P \wedge Q$, we may derive Q .

$\vee$ Introduction Left

Introduces a disjunction by adding an additional possibility.

From P , we may derive $P \vee Q$.

$\vee$ Introduction Right

Introduces a disjunction using the right operand.

From Q , we may derive $P \vee Q$.

$\vee$ Elimination

Allows reasoning by cases. If $P \vee Q$ is known, and both P and Q independently lead to the same conclusion R , then R may be derived.

$\rightarrow$ Introduction

Derives an implication by discharging a subproof.
If assuming P leads to Q , we may conclude $P \rightarrow Q$.

$\rightarrow$ Elimination (Modus Ponens)

Applies an implication to derive its consequence.
From $P \rightarrow Q$ and P , we may derive Q .

$\neg$ Introduction

Derives a negation by showing that assuming a statement leads to a contradiction.

$\neg$ Elimination

If both P and $\neg P$ are derived, a contradiction is produced.

$\leftrightarrow$ Introduction

Derives a biconditional by proving both directions.
If $P \rightarrow Q$ and $Q \rightarrow P$ are both shown, we may conclude $P \leftrightarrow Q$.

$\leftrightarrow$ Elimination Left

Uses a biconditional to derive one direction of implication.
From $P \leftrightarrow Q$, we may derive $P \rightarrow Q$.

$\leftrightarrow$ Elimination Right

Uses a biconditional to derive the opposite implication.

Uses a biconditional to derive the opposite implication.
From $P \leftrightarrow Q$, we may derive $Q \rightarrow P$.

9. System Feedback Messages

The interface provides feedback messages to guide the user.

Examples include:

Brewing levels...

Indicates the proof checking process has started.

No problems available for this mode. Check your JSON ranges.

Occurs when no exercises are available.

 Add ingredients first.

The user attempted to add a proof line without creating a formula.

 Invalid formula for this mode.

The formula contains symbols not allowed in the selected logic mode.

Added to spellbook.

Confirms that the formula was successfully added as a proof line.

 Write the assumption formula first.

An assumption was attempted without specifying a formula.

 Invalid assumption.

The assumption formula is not valid.

 No open subproof.

The user attempted to close a subproof that does not exist.

 Add lines before brewing!

The proof cannot be checked because no lines have been added.

 You must derive the result, not assert it.

The user attempted to state the goal without deriving it.

Checking potion stability...

Indicates the backend is verifying the proof.

 Potion Complete! Your proof is correct.

The proof is valid.

 Potion Failed. Incorrect proof.

The proof contains errors.

 Backend not running

The frontend cannot connect to the backend server.

 Checking with the Oracle...

Indicates that the proof is being evaluated.

10. Tutorial Overlay

When the system is opened for the first time, a tutorial overlay automatically appears.

The tutorial highlights each part of the interface and explains its purpose. This helps new users quickly understand how to use the system.

The tutorial guides the user through:

- The workspace

- Ingredient buttons
- The Spellbook
- Rule selection
- Proof validation

The Tutorial button can be used to reopen the tutorial at any time.

11. README Information

The project repository includes a README file containing instructions for running the system locally.

The README is intended for:

- Project markers
- Developers
- Users downloading the repository

The README provides:

- Setup instructions
- Dependency requirements
- Commands required to run the frontend and backend components

This ensures that anyone accessing the repository can easily run and understand the DeductIT system.